

Guía de Desarrollo: Marca Personal FP

Para: Alumnado de 2º DAW · Nivel medio en Laravel **Cómo usar esta guía:** En cada apartado verás **3 secciones fijas**:

- **¿Qué es?** → Teoría mínima.
- **Ejemplo genérico** → Código de muestra (tienda de **Videojuegos**). **NO lo copies al proyecto.**
- **Reto del proyecto** → Lo que TÚ programas. Te doy comandos y campos, **no el código.**

Nota sobre los datos: Los nombres de tablas y campos siguen el modelo estándar de *Marca Personal FP*. **Verifica los nombres exactos** en la documentación de tu repositorio (**documentos/0613_Servidor**) por si tu curso usa alguna variante.

BLOQUE 1: Configuración Inicial y Autenticación

1.1. Configuración del proyecto y **.env**

¿Qué es y para qué sirve?

- El archivo **.env** guarda la **configuración secreta** de tu proyecto: nombre de la app, conexión a base de datos, claves, etc.
- **Nunca se sube a Git** (está en **.gitignore**). Cada desarrollador tiene el suyo.
- Laravel lee estos valores con la función **env()** y los cachea en los archivos de **config/**.

Ejemplo genérico (tienda de Videojuegos)

Creación del proyecto y configuración mínima del **.env**:

Shell

```
composer create-project laravel/laravel tienda-videojuegos
cd tienda-videojuegos
```

None

```
APP_NAME="Tienda Videojuegos"
APP_ENV=local
APP_DEBUG=true
APP_URL=http://localhost:8000

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=tienda_videojuegos
DB_USERNAME=root
DB_PASSWORD=secret
```

Crear la base de datos y arrancar:

Shell

```
php artisan key:generate
php artisan serve
```

Reto del proyecto

1. **Crea el proyecto** llamado `marcapersonalFP`.
2. **Crea la base de datos** en MySQL (ej. `marcapersonalfp`).
3. **Configura tu `.env`** con estos valores:

Variable	Valor que debes poner
APP_NAME	"Marca Personal FP"
APP_URL	Tu URL local (ej. <code>http://localhost:8000</code>)
DB_CONNECTION	mysql
DB_DATABASE	Nombre de tu BD
DB_USERNAME	Tu usuario MySQL

DB_PASSWORD Tu contraseña MySQL
D

4. **Genera la clave** y comprueba que arranca:

Shell

```
php artisan key:generate  
php artisan serve
```

Objetivo: Ver la página de bienvenida de Laravel sin errores de conexión.

1.2. Instalación de Laravel Breeze

¿Por qué **AHORA** y no al final? Breeze **genera rutas, vistas y un layout base**. Si lo instalas tarde, **machaca** tus rutas y vistas propias. Instalándolo ya, partimos de una base limpia sobre la que construir.

¿Qué es y para qué sirve?

- Breeze es un *starter kit* de autenticación. Te da **login, registro, recuperar contraseña y perfil** ya hechos.
- Crea las **rutas** (`auth.php`), las **vistas Blade** y un **layout** (`layouts/app.blade.php`).
- Trae el **middleware** `auth` listo para proteger rutas.

Ejemplo genérico (tienda de Videojuegos)

Shell

```
composer require laravel/breeze --dev  
php artisan breeze:install blade  
npm install && npm run dev  
php artisan migrate
```

- `breeze:install blade` → instala la versión con **Blade** (sin React/Vue).
- `npm run dev` → compila CSS/JS (Tailwind).
- `migrate` → crea las tablas `users`, `sessions`, etc.

Aviso (Laravel 12): El instalador `laravel new` ya no ofrece Breeze en su menú; ahí debes elegir **"None"**. Breeze **sigue siendo compatible**: se instala después, manualmente, con los comandos de arriba (`composer require laravel/breeze --dev + breeze:install`). Si creas el proyecto con

`composer create-project laravel/laravel`, no verás ese menú y vas directo a instalar Breeze.

Reto del proyecto

1. **Instala Breeze** en `marcapersonalFP` usando la **stack Blade**.
2. **Compila** los assets e **inicia** las migraciones base.

Comandos a usar (mismos que el ejemplo):

```
Shell
composer require laravel/breeze --dev
php artisan breeze:install blade
npm install && npm run dev
php artisan migrate
```

3. **Comprueba** que funciona:

Ruta	Qué debe pasar
<code>/register</code>	Puedo registrarme
<code>/login</code>	Puedo iniciar sesión
<code>/dashboard</code>	Solo accesible logueado

Objetivo: Registro y login operativos **antes** de tocar la base de datos del proyecto.

BLOQUE 2: Capa de Datos

2.1. Migraciones

¿Qué es y para qué sirve?

- Una **migración** es el "control de versiones" de tu base de datos: define **tablas y columnas en PHP**.
- Permite **crear y recrear** la BD con un comando, sin SQL manual.
- Cada migración tiene `up()` (crear) y `down()` (deshacer).

Ejemplo genérico (tienda de Videojuegos)

Shell

```
php artisan make:migration create_videojuegos_table
```

PHP

```
public function up(): void
{
    Schema::create('videojuegos', function (Blueprint $table) {
        $table->id();
        $table->string('titulo');
        $table->text('descripcion')->nullable();
        $table->decimal('precio', 8, 2);
        $table->integer('stock')->default(0);
        $table->enum('plataforma', ['PC', 'PS5', 'Xbox',
'Switch']);
        $table->boolean('disponible')->default(true);
        $table->string('slug')->unique();
        $table->timestamps();
    });
}
```

Tipos de columna habituales: `string`, `text`, `integer`, `decimal`, `boolean`, `enum`, `date`, `foreignId`.

Reto del proyecto

Crea **una migración por cada tabla** del proyecto. Comando:

Shell

```
php artisan make:migration create_NOMBRE_table
```

Tablas y campos a implementar (tú decides el tipo exacto según la teoría):

familias_profesionales

Campo	Tipo sugerido	Notas
id	id()	PK

codigo	string	único
nombre	string	
slug	string	único (para URLs)
timestamps	timestamps()	

ciclos_formativos

Campo	Tipo sugerido	Notas
id	id()	PK
codigo	string	único
nombre	string	
grado	enum	medio / superior
slug	string	único
familia_profesional_id	foreignId	FK → familias_profesionales
timestamps	timestamps()	

estudiantes

Campo	Tipo sugerido	Notas
id	id()	PK
user_id	foreignId	FK → users, nullable
dni	string	único
nombre	string	
apellidos	string	

email	string	único
telefono	string	nullable
direccion	string	nullable
imagen	string	nullable (ruta foto)
slug	string	único
timestamps	timestamps()	

docentes

Campo	Tipo sugerido	Notas
id	id()	PK
user_id	foreignId	FK → <code>users</code>
nombre	string	
apellidos	string	
email	string	único
slug	string	único
timestamps	timestamps()	

proyectos

Campo	Tipo sugerido	Notas
id	id()	PK
nombre	string	
descripcion	text	

<code>url</code>	<code>string</code>	nullable (repo/demo)
<code>imagen</code>	<code>string</code>	nullable
<code>dificultad</code>	<code>enum o string</code>	ej. <code>baja/media/alta</code>
<code>slug</code>	<code>string</code>	único
<code>timestamps</code>	<code>timestamps()</code>	

Regla: El campo `_id` debe ir **después** de que exista la tabla a la que apunta (orden de migraciones).

Objetivo: `php artisan migrate` sin errores.

2.2. Relaciones y Tablas Pivot

¿Qué es y para qué sirve?

- Una **tabla pivot** conecta dos tablas en una relación **muchos-a-muchos (N:M)**.
- No tiene modelo propio: solo guarda las **claves foráneas** de las dos tablas que une.
- **Convención de nombre:** los dos modelos en **singular, orden alfabético**, separados por `_`.
 - Ej.: `categoria + videojuego` → `categoria_videojuego`

Ejemplo genérico (tienda de Videojuegos)

Un videojuego puede tener **varias categorías** y una categoría **varios videojuegos** (N:M):

Shell

```
php artisan make:migration create_categoria_videojuego_table
```

PHP

```
Schema::create('categoria_videojuego', function (Blueprint
$table) {
    $table->id();

    $table->foreignId('categoria_id')->constrained()->cascadeOnDelete
());
```



```
$table->foreignId('videojuego_id')->constrained()->cascadeOnDelete();
    $table->timestamps();
});
```

Reto del proyecto

Relaciones del proyecto:

Relación	Tipo	Cómo se resuelve
Familia → Ciclos	1:N	FK <code>familia_profesional_id</code> (ya está en 2.1)
Ciclo ↔ Estudiantes	N:M	Tabla pivot
Estudiante ↔ Proyectos	N:M	Tabla pivot
Docente ↔ Proyectos	N:M	Tabla pivot
User → Estudiante / Docente	1:1	FK <code>user_id</code> (ya está en 2.1)

Tablas pivot a crear (recuerda: singular + alfabético):

Pivot	Claves foráneas
<code>ciclo_formativo_estudiante</code>	<code>ciclo_formativo_id</code> , <code>estudiante_id</code>
<code>estudiante_proyecto</code>	<code>estudiante_id</code> , <code>proyecto_id</code>
<code>docente_proyecto</code>	<code>docente_id</code> , <code>proyecto_id</code>

Comando para cada una:

```
Shell
php artisan make:migration create_NOMBRE_PIVOT_table
```

Pista: Usa `->constrained()` para crear la FK automáticamente y `->cascadeOnDelete()` si quieres que se borre el vínculo al borrar el registro.

Objetivo: `migrate:fresh` crea todas las tablas + pivots sin errores.

2.3. Factories

¿Qué es y para qué sirve?

- Una **factory** define **cómo generar datos falsos** y realistas para una tabla.
- Usa la librería **Faker** (`fake()`): nombres, emails, frases, números aleatorios.
- Sirve para **rellenar la BD de pruebas** sin escribir datos a mano.

Ejemplo genérico (tienda de Videojuegos)

Shell

```
php artisan make:factory VideojuegoFactory
```

PHP

```
public function definition(): array
{
    return [
        'titulo' => fake()->words(3, true),
        'descripcion' => fake()->paragraph(),
        'precio' => fake()->randomFloat(2, 5, 70),
        'stock' => fake()->numberBetween(0, 100),
        'plataforma' => fake()->randomElement(['PC', 'PS5',
'Xbox', 'Switch']),
        'disponible' => fake()->boolean(),
        'slug' => fake()->unique()->slug(),
    ];
}
```

Reto del proyecto

Crea una **factory por modelo principal**. Comando:

Shell

```
php artisan make:factory NombreFactory
```

Factories a crear y pistas de Faker:

Factory

Campos clave y Faker sugerido

FamiliaProfesionalFactory	codigo → fake()->unique()->bothify(), nombre, slug
CicloFormativoFactory	grado → randomElement(['medio', 'superior']), FK familia
EstudianteFactory	dni → unique(), nombre, apellidos, email → unique()->safeEmail()
DocenteFactory	nombre, apellidos, email → unique()->safeEmail()
ProyectoFactory	nombre, descripcion → paragraph(), dificultad → randomElement(...)

Pistas importantes:

- Para las **FK** dentro de una factory, usa: 'familia_profesional_id' => FamiliaProfesional::factory().
- Para campos **únicos** (dni, slug, email), añade siempre ->unique().

Objetivo: Cada factory genera 1 registro válido sin chocar con campos únicos.

2.4. Seeders

¿Qué es y para qué sirve?

- Un **seeder** **llena la base de datos** llamando a las factories (o insertando datos fijos).
- El **DatabaseSeeder** es el **director**: decide **qué seeders** se ejecutan y en **qué orden**.
- **El orden importa**: primero las tablas "padre" (sin FK), luego las que dependen de ellas.

Ejemplo genérico (tienda de Videojuegos)

Shell

```
php artisan make:seeder VideojuegoSeeder
```

PHP

```
// VideojuegoSeeder.php
public function run(): void
```

```
{  
    Videojuego::factory(20)->create();  
}
```

PHP

```
// DatabaseSeeder.php  
public function run(): void  
{  
    $this->call([  
        VideojuegoSeeder::class,  
    ]);  
}
```

Ejecutar:

Shell

```
php artisan db:seed  
# o reconstruir todo desde cero:  
php artisan migrate:fresh --seed
```

Reto del proyecto

1. **Crea un seeder por entidad** con `php artisan make:seeder NombreSeeder`.
2. **Registra el orden correcto** en `DatabaseSeeder`.

Orden de ejecución obligatorio (de "padre" a "hijo"):

Orden	Seeder	Por qué va aquí
1	<code>FamiliaProfesionalSeeder</code>	No depende de nadie
2	<code>CicloFormativoSeeder</code>	Necesita familias ya creadas
3	<code>EstudianteSeeder</code>	Independiente / enlaza con users
4	<code>DocenteSeeder</code>	Independiente / enlaza con users
5	<code>ProyectoSeeder</code>	Se relaciona con estudiantes y docentes

3.

Rellena las pivots: dentro de los seeders de proyecto/estudiante usa `attach()` o `->hasAttached()` para vincular N:M.

4. **Reconstruye y comprueba:**

Shell

```
php artisan migrate:fresh --seed
```

Objetivo: La BD se llena entera sin errores de clave foránea y las pivots tienen vínculos.

Has terminado Bloques 1 y 2. Ya tienes: proyecto configurado, autenticación, todas las tablas, relaciones, datos falsos y BD poblada. El siguiente paso (Bloque 3) será **dar vida a esos datos con los Modelos Eloquent y la capa de Servicios.**